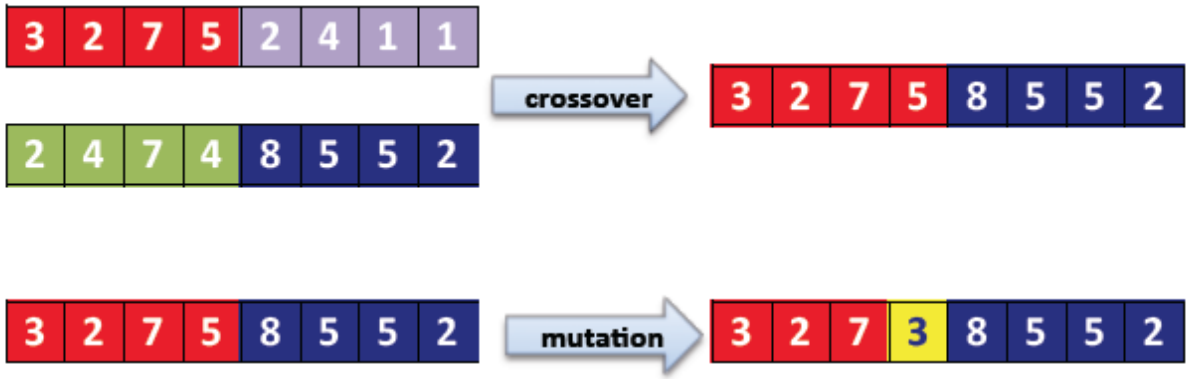
# CSE358

## ARTIFICIAL INTELLIGENCE

### 04. Search Agents & Local Search

# Outline

- Local Search
- Local Search Algorithms
  - Hill Climbing
  - Simulated Annealing
  - Local Beam Search
  - Genetic Algorithms



# Local Search

- Search algorithms seen so far are designed to **explore search spaces systematically**
- **Problems:** observable, deterministic, known environments where the solution is a sequence of actions
- Real-World problems are more complex
- When a goal is found, the path to that goal constitutes a solution to the problem. But, depending on the applications, the path may or may not matter
- If the path does not matter/systematic search is not possible, then consider another class of algorithms → **Local Search**

# Local Search

- In such cases, we can use iterative improvement algorithms, **Local Search**
- Also useful in pure **optimization problems** where the goal is to find the best state according to an **objective function**
- ***Examples***
  - Integrated circuit design, telecommunications network optimization, airline flight scheduling, etc.
  - *N-puzzle or 8-queen*: what matters is the final configuration of the puzzle, not the intermediary steps to reach it
  - Darwinian evolution: no «goal test» and no «path cost» but the objective function of «reproductive fitness»

# Local Search

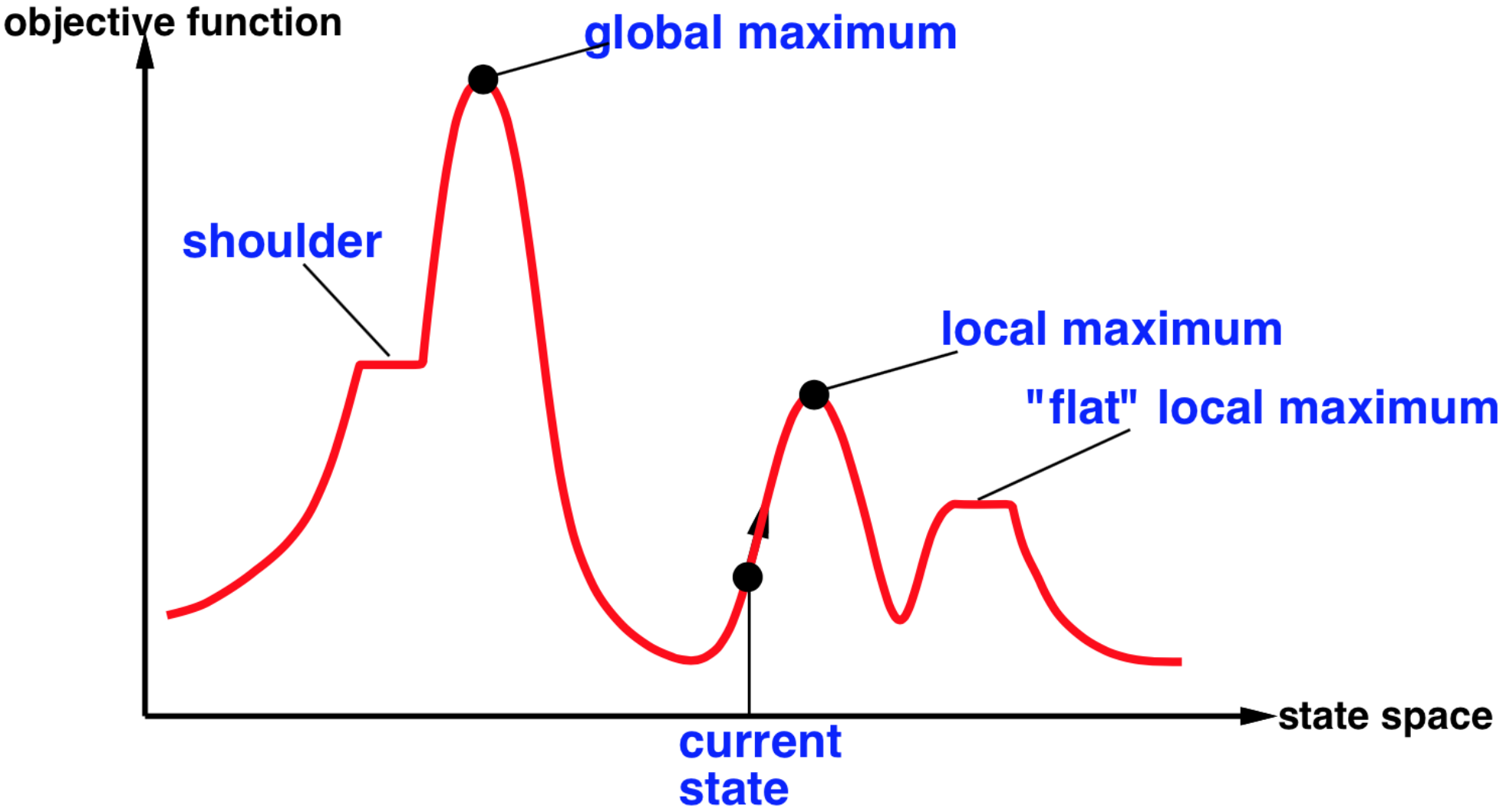
- ***Idea:*** **keep a single "current" state** *(rather than multiple paths)***, and try to improve it**
- Move only to the neighbors of that node
- ***Advantages***
  1. No need to maintain a search tree
  2. Use very little memory
  3. Can often find good enough solutions in continuous or large state spaces

# Local Search

## Local Search Algorithms

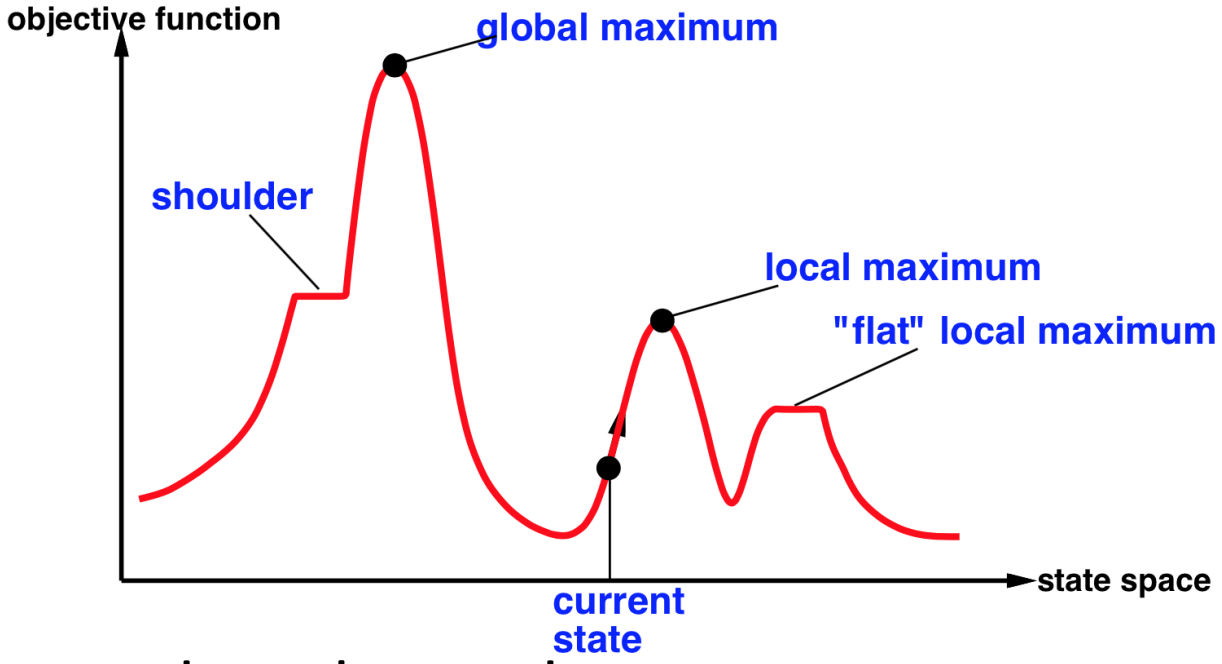
1. **Hill Climbing** (steepest ascent / descent)
2. **Simulated Annealing**: inspired by statistical physics
3. **Local Beam Search**
4. **Genetic Algorithms**: inspired by evolutionary biology

# State Space Landscape for Local Search




State Space Landscape

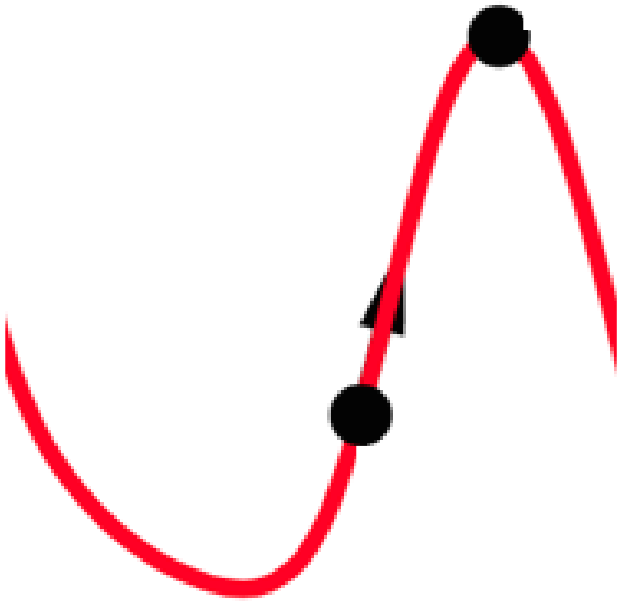
# 1. Hill Climbing




- Also called **Greedy Local Search**
- Looks only to immediate good neighbors and not beyond
- Search moves uphill:
  - moves in the direction of increasing elevation/value to find the top of the mountain
- Terminates when it reaches a **peak**
- Can terminate with a local maximum, global maximum or can get stuck and no progress is possible
- A **node** is **a state** and **a value**

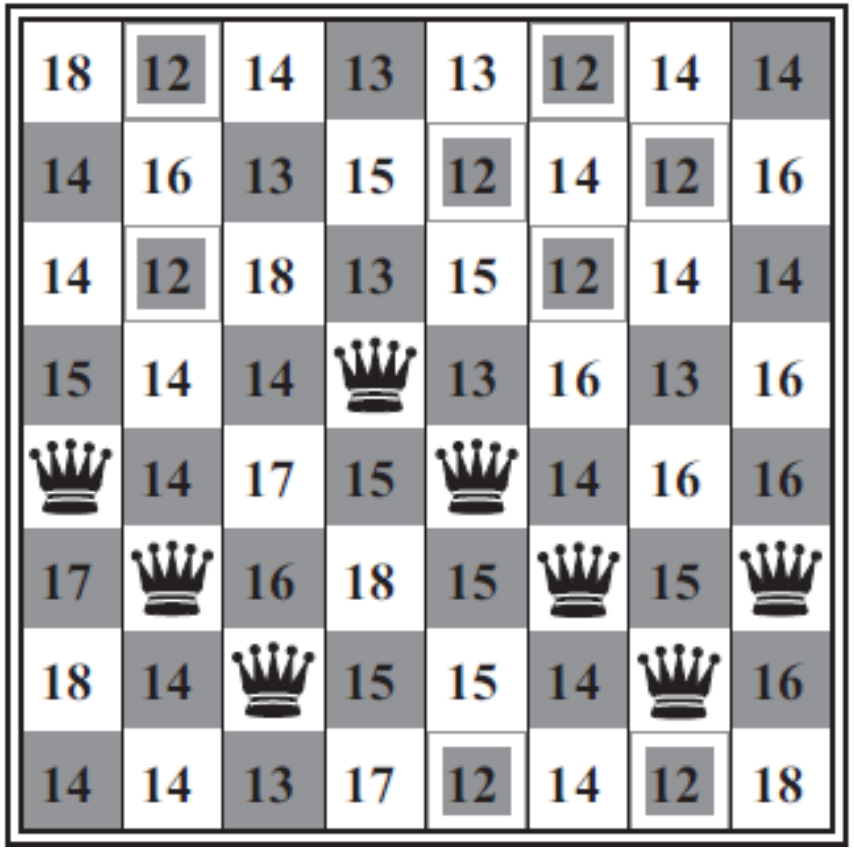
***"Like climbing Everest in thick fog with amnesia"***

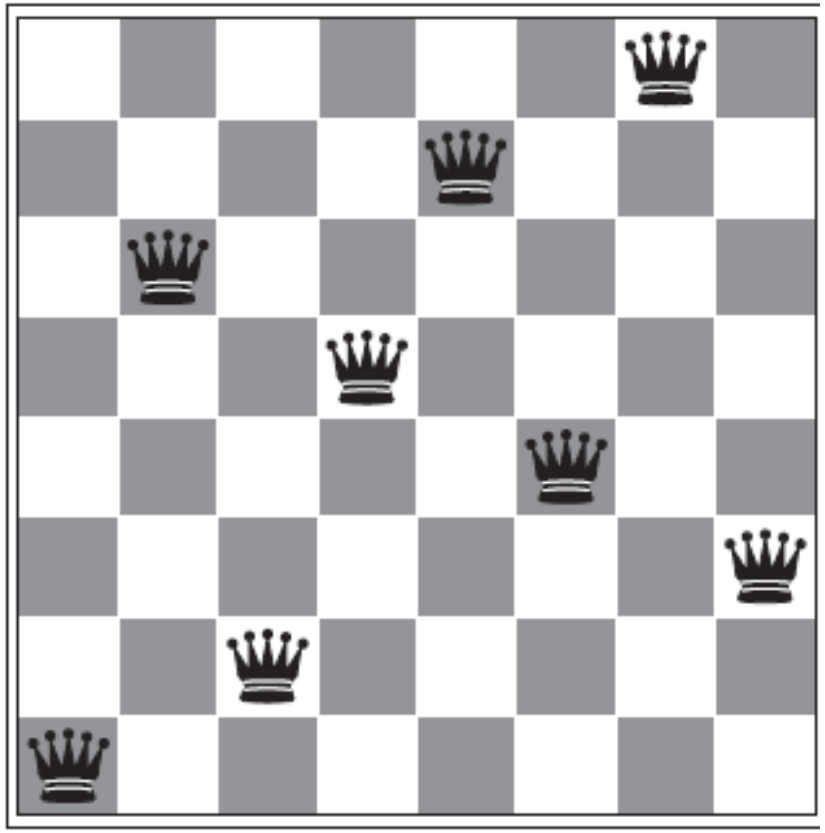
# 1. Hill Climbing



```
function HILL-CLIMBING(initialState)
    returns State that is a local maximum

    initialize current with initialState

    loop do
        neighbor = a highest-valued successor of current

        if neighbor.value ≤ current.value:
            return current.state

        current = neighbor
```

# 1. Hill Climbing (8-queens example)



$h$: number of pairs of queens that are attacking each other (here $h = 17$)

State formulation: 8 queens one per column

Successors: all possible states generated by moving a single queen in its column ($8 \times 7 = 56$ successors)

A local minimum in the 8-queens state space; the state has $h = 1$ but every successor has a higher cost

# 1. Hill Climbing

A flat area of the state-space landscape (such as a flat local maximum from which no uphill exits, or a shoulder)

- Other variants of hill climbing include
  - **Sideways moves** escape from **plateaux** where best successor has same value as the current state
  - **Random-restart hill climbing** overcomes local maxima: keep trying! (either find a goal or get several possible solution and pick the max)
  - **Stochastic hill climbing** chooses at random among the uphill moves

- **Hill climbing** effective in general but depends on shape of the landscape
- Successful in many real-problems after a reasonable number of restarts
  - 8-queens: 86% stuck, 14% solve, 4-steps average on success, 3-steps average on stuck, not bad for a state space with $8^8 \approx 17$ million states.

# 2. Simulated Annealing

- **Idea: Escape local maxima by allowing "downhill" moves**
  - But make them rarer as time goes on
- Combine hill climbing with a random walk in some way that yields both efficiency and completeness
- In metallurgy, **annealing** is the process used to temper or harden metals and glass by heating them to a high temperature and then gradually cooling them, thus allowing the material to reach a low-energy crystalline state
- Imagine the task of getting a ping-pong ball into the deepest crevice in a bumpy surface
  - If we just let the ball roll, it will come to rest at a local minimum
  - If we shake the surface, we can bounce the ball out of the local minimum.
  - The trick is to shake just hard enough to bounce the ball out of local minima but not hard enough to dislodge it from the global minimum
  - The simulated-annealing solution is to start by shaking hard (i.e., at a high temperature) and then gradually reduce the intensity of the shaking (i.e., lower the temperature)

# 3. Local Beam Search

- **Local beam search** keeps track of ***k*** states rather than one single state
  - Initially: Begin with ***k*** randomly generated states
  - Next: all the successors of all ***k*** states are generated
  - If any of successors is goal → finished
  - Else **select *k* best successors** from the complete list and repeat

- **Major difference with random-restart search**
  - In a random-restart search, each search process runs independently of the others
  - In a local beam search, useful information is passed among the parallel search threads (states)

- **Can suffer from lack of diversity among the *k* states** — they can quickly become concentrated in a small region (around the same part) of the state space
  - Help alleviate the problem of the states agglomerating, we can use **Stochastic Beam Search**, where we **choose *k* successors at random**

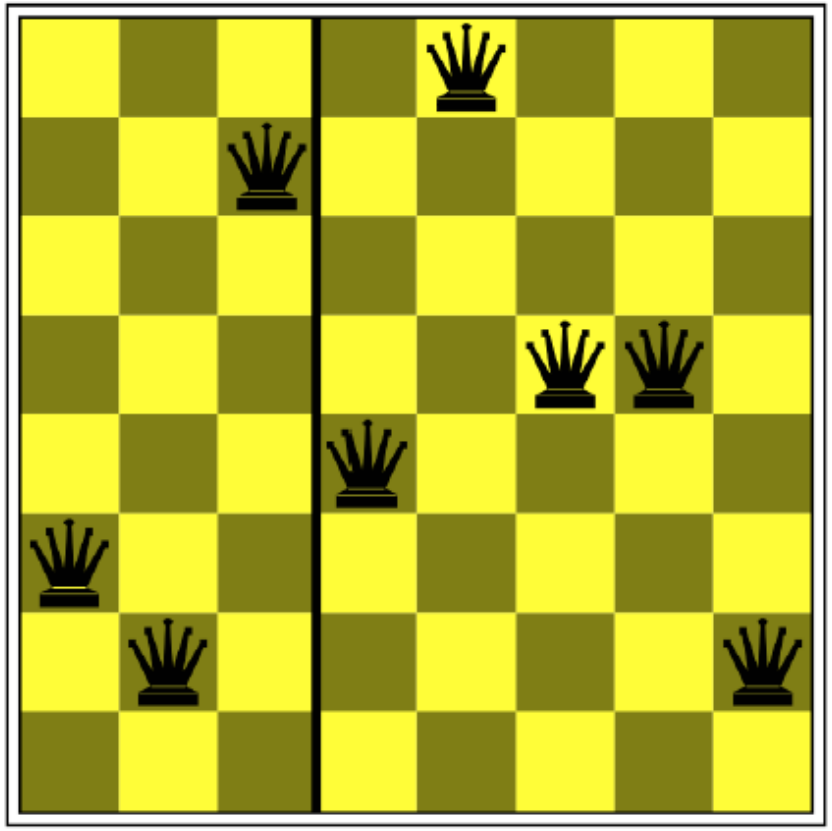
# 4. Genetic Algorithms

- **Genetic algorithms (GA)** is **a variant of stochastic beam search**
- Successor states are generated by combining ***two*** parents rather by modifying a single state
- The process is inspired by **natural selection**
  - Same as in stochastic beam search, except that now deal with sexual rather than asexual reproduction
- Starts with ***k* randomly generated states**, called **population**
- Each state is an **individual**
- An individual is usually represented by a **string** of **0**'s and **1**'s, or digits, in general a finite set
- The objective function is called **fitness function**: **better states have high values of fitness function**

# 4. Genetic Algorithms

- In the 8-queen problem, an individual can be represented by a string **digits** 1 to 8, that represents the position of the 8 queens in the 8 columns

**[ 3 2 7 4 8 5 5 2 ]**

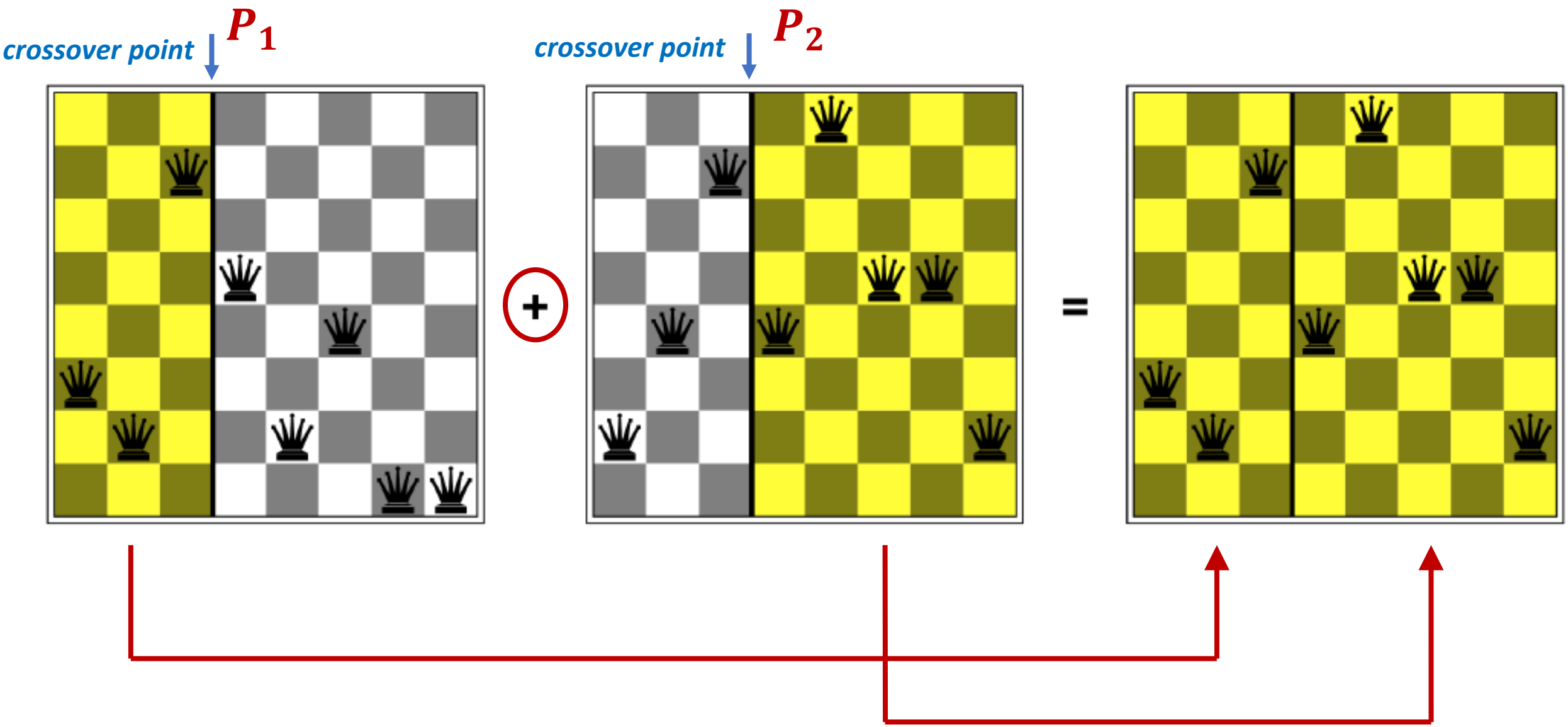


# 4. Genetic Algorithms

***The objective function is called fitness function: better states have high values of fitness function***

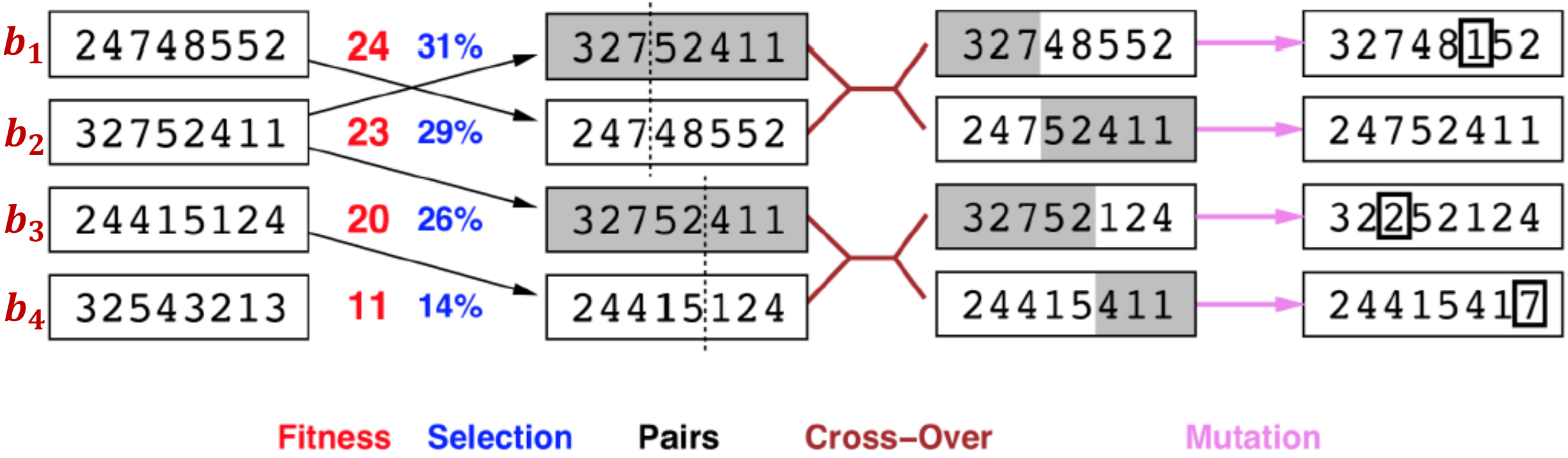
- Possible fitness function is **the number of non-attacking pairs of queens**
- Fitness function of the solution: 7+6+5+4+3+2+1 = **28**
- Pairs of individuals are selected at random for **reproduction** with respect to some probabilities
- A **crossover** point is chosen randomly in the string
- **Offspring** are created by crossing the parents at the crossover point
- Each element in the string is also subject to some **mutation** with a small probability

# 4. Genetic Algorithms: Crossover



# 4. Genetic Algorithms

Generate successors from pairs of states



# 4. Genetic Algorithms

## The Basic Genetic Algorithm

1. Generate random population
2. Until the end condition is met, create a new population by repeating following steps
   - Evaluate the **fitness function**
   - **Select two parents** from a population, weighed by their fitness function values
   - With probability $p_c$ **cross over the parents** to form a new offspring
   - With probability $p_m$ **mutate new offspring**
   - Place new offspring in the new population
3. Return ***the best solution in current population***

# 4. Genetic Algorithms

```
function GENETIC-ALGORITHM(population, fitness-function)
    returns an individual

    repeat

        initialize new-population with ∅

        for i=1 to size(population) do

            x = random-select(population,fitness-function)
            y = random-select(population,fitness-function)
            child = cross-over(x,y)
            mutate (child) with a small random probability
            add child to new-population

        population = new-population

    until some individual is fit enough or enough time has elapsed

    return the best individual in population w.r.t. fitness-function
```

# Summary

- So far, we keep the paths to the goal!
- For some problems (like 8-queens) we don't care about the path, we only care about the solution - Many real problems are of this form
- **Local search** algorithms operate using a single **current state** and generally move to neighbors of that state
- There is an **objective function** that tells the value of each state, and the goal has the highest value (global maximum)
- Algorithms like **Hill Climbing** try to move to a neighbour with the highest value
- Danger of being stuck in a **local maximum**. So some randomness is added to "shake" out of local maxima
- **Simulated Annealing**: Instead of the best move, take a random move and if it improves the situation then always accept, otherwise accept with a probability < 1
- A **genetic algorithm** is a stochastic hill-climbing search in which a large population of states is maintained
  - New states are generated by **mutation** and by **crossover**, which combines pairs of states from the population